

SignatureValidator Component

Langflow custom component — validates symbols used in AST constraints against the declared z3_signature.

```
from langflow.custom import Component
from langflow.io import MessageTextInput, Output
from langflow.schema.message import Message

import json


class SignatureValidator(Component):

    display_name = "Signature Validator"
    name = "SignatureValidator"
    description = (
        "Validates symbols used in AST constraints against "
        "the declared z3_signature"
    )

    inputs = [
        MessageTextInput(
            name="input_value",
            display_name="Formalized AST"
        )
    ]

    outputs = [
        Output(
            name="output",
            display_name="Validated AST",
            method="build_output"
        )
    ]

    def collect_symbols(self, node, symbols):

        if not isinstance(node, dict):
            return

        if node.get("type") == "VAR":

            symbol_name = node.get("name")

            if symbol_name:
                symbols.add(symbol_name)

        if "operand" in node:
            self.collect_symbols(
                node["operand"],
                symbols
            )

        if "antecedent" in node:
            self.collect_symbols(
                node["antecedent"],
```

```

        symbols
    )

    if "consequent" in node:
        self.collect_symbols(
            node["consequent"],
            symbols
        )

    if "left" in node:
        self.collect_symbols(
            node["left"],
            symbols
        )

    if "right" in node:
        self.collect_symbols(
            node["right"],
            symbols
        )

    for child in node.get(
        "operands",
        []
    ):
        self.collect_symbols(
            child,
            symbols
        )

def build_output(self) -> Message:

    try:

        raw = (
            self.input_value.text
            if hasattr(
                self.input_value,
                "text"
            )
            else str(self.input_value)
        )

        data = json.loads(raw)

    except Exception as e:

        return Message(
            text=json.dumps(
                {
                    "signature_valid": False,
                    "error": str(e)
                },
                indent=2,
                ensure_ascii=False
            )
        )

```

```

validation_errors = []

requirements = data.get(
    "requirements",
    []
)

for req in requirements:
    declared_symbols = set()

    z3_signature = req.get(
        "z3_signature",
        {}
    )

    for predicate in z3_signature.get(
        "predicates",
        []
    ):
        name = predicate.get("name")

        if name:
            declared_symbols.add(name)

    for constant in z3_signature.get(
        "constants",
        []
    ):
        name = constant.get("name")

        if name:
            declared_symbols.add(name)

    used_symbols = set()

    for ast in req.get(
        "ast_constraints",
        []
    ):
        self.collect_symbols(
            ast,
            used_symbols
        )

    undeclared_symbols = sorted(
        used_symbols -
        declared_symbols
    )

    if undeclared_symbols:
        validation_errors.append(

```

```

        {
            "requirement_id":
                req.get(
                    "requirement_id",
                    "UNKNOWN"
                ),

            "error":
                "UNDECLARED_SYMBOLS",

            "symbols":
                undeclared_symbols
        }
    )

result = {
    "signature_valid":
        len(validation_errors) == 0,

    "errors":
        validation_errors,

    "requirements":
        requirements
}

return Message(
    text=json.dumps(
        result,
        indent=2,
        ensure_ascii=False
    )
)

```